피어 인증 과정 분석 (Peer Authentication Process Analysis)

개요

이 문서는 Rust libp2p에서 피어 간 상호 인증이 어떻게 이루어지는지에 대한 상세한 분석을 제공합니다. libp2p의 피어 인증은 보안 계층(Noise Protocol, TLS)을 통해 구현되며, 각 피어의 신원을 암호학적으로 검증하는 과정을 포함합니다.

피어 신원 체계 (Peer Identity System)

PeerId 생성 과정

```
// identity/src/peer_id.rs

impl PeerId {

pub  fn from_public_key(key: &crate::keypair::PublicKey) -> PeerId {

let key_enc = key.encode_protobuf();



let multihash = if key_enc.len() <= MAX_INLINE_KEY_LENGTH {

// 작은 키는 identity multihash 사용 (직접 포함)

Multihash::wrap(MULTIHASH_IDENTITY_CODE, &key_enc)

} else {

// 큰 키는 SHA256 해시 사용

Multihash::wrap(MULTIHASH_SHA256_CODE, &sha2::Sha256::digest(key_enc))
```

```
};


PeerId { multihash }

}

}
```

핵심 특징:

- PeerId는 공개키의 multihash로 생성됩니다
- 42바이트 이하의 작은 키는 identity multihash로 직접 포함
- 큰 키는 SHA256 해시를 사용하여 고정 크기로 변환
- CIDv0 호환 형식으로 네트워크 간 상호 운용성 보장

지원되는 키 타입

```rust
// identity/src/lib.rs

pub enum KeyType {

Ed25519, // 권장: 빠르고 안전한 타원곡선 암호

RSA, // 레거시 지원

Secp256k1, // 비트코인/이더리움 호환

Ecdsa, // NIST 표준 타원곡선

}
```

Noise Protocol을 통한 피어 인증

XX 핸드셰이크 패턴

Noise Protocol의 XX 패턴은 상호 인증을 제공하는 핸드셰이크입니다:

```
Initiator Responder

| |

| -> e | (ephemeral key)

| <- e, ee, s, es | (ephemeral + static keys)

| -> s, se | (static key exchange)

| |
```

인증 과정 상세 분석

1. 키 쌍 생성 및 인증

```rust
// transports/noise/src/protocol.rs

impl Keypair {

pub(crate) fn into_authentic(

self,

id_keys: &identity::Keypair,

) -> Result<AuthenticKeypair, Error> {

// DH 공개키에 대한 서명 생성

let sig = id_keys.sign(&[STATIC_KEY_DOMAIN.as_bytes(), self.public.as_ref()].con
```

```rust
let identity = KeypairIdentity {
```

```
public: id_keys.public(),

signature: sig,

};



Ok(AuthenticKeypair {

keypair: self,

identity,

})

}

}
```

핵심 요소:

- DH 키 쌍은 임시적이며 각 연결마다 새로 생성
- Identity 키로 DH 공개키에 서명하여 인증성 보장
- 도메인 분리(`STATIC_KEY_DOMAIN`)로 서명 재사용 공격 방지

2. 핸드셰이크 메시지 교환

```
// transports/noise/src/io/handshake.rs

pub(crate) async fn send_identity<T>(state: &mut State<T>) -> Result<(), Error>

where

T: AsyncRead + AsyncWrite + Unpin,

{

let mut pb = proto::NoiseHandshakePayload {

identity_key: state.identity.public.encode_protobuf(),
```

```
..Default::default()
};

pb.identity_sig.clone_from(&state.identity.signature);
// WebTransport 확장 지원
if state.io.codec().is_responder() {
if let Some(ref certhashes) = state.responder_webtransport_certhashes {
let ext = pb.extensions.get_or_insert_with(proto::NoiseExtensions::default);
ext.webtransport_certhashes = certhashes.iter().map(|hash| hash.to_bytes()).coll
}
}

state.io.send(&pb).await?;
Ok(())
}
```

3. 서명 검증 과정

```
// transports/noise/src/io/handshake.rs
impl<T> State<T> {
pub(crate) fn finish(self) -> Result<(identity::PublicKey, Output<T>), Error> {
let (pubkey, framed) = map_into_transport(self.io)?;
```

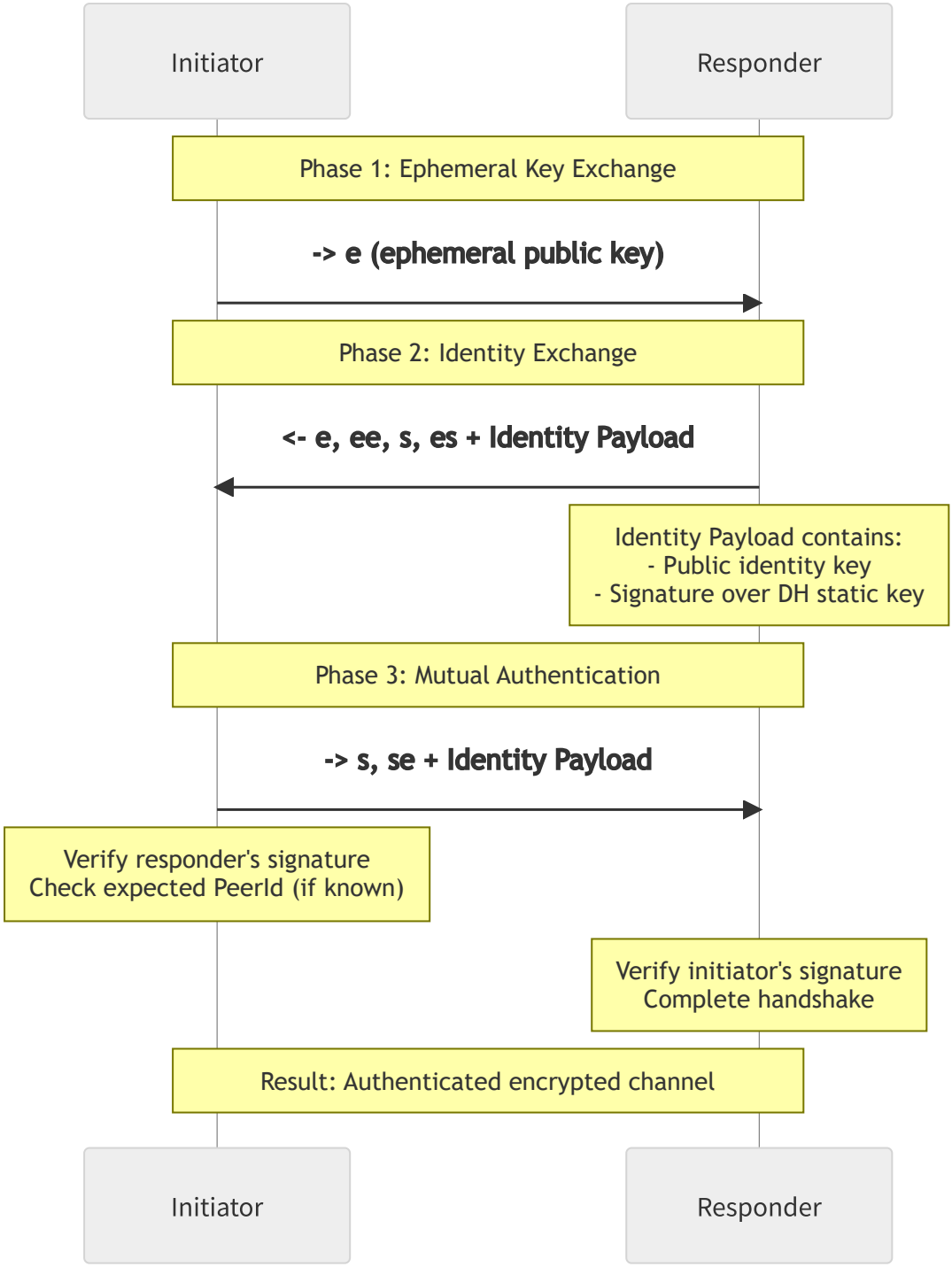
```
let id_pk = self.id_remote_pubkey
.ok_or_else(|| Error::AuthenticationFailed)?;


// DH 공개키에 대한 서명 검증
let is_valid_signature = self.dh_remote_pubkey_sig.as_ref().is_some_and(|s| {
id_pk.verify(&[STATIC_KEY_DOMAIN.as_bytes(), pubkey.as_ref()].concat(), s)
});


if !is_valid_signature {
return Err(Error::BadSignature);
}


Ok((id_pk, Output::new(framed)))
}
}
```

핸드셰이크 흐름도



TLS를 통한 피어 인증

libp2p TLS 확장

TLS 기반 인증은 X.509 인증서에 libp2p 확장을 포함하여 구현됩니다:

```
// transports/tls/src/verifier.rs
```

```rust
impl ServerCertVerifier for Libp2pCertificateVerifier {
fn verify_server_cert(
&self,
end_entity: &CertificateDer,
intermediates: &[CertificateDer],
_server_name: &rustls::pki_types::ServerName,
_ocsp_response: &[u8],
_now: rustls::pki_types::UnixTime,
) -> Result<ServerCertVerified, rustls::Error> {
let peer_id = verify_presented_certs(end_entity, intermediates)?;


// PeerId 검증
if let Some(remote_peer_id) = self.remote_peer_id {
if remote_peer_id != peer_id {
return Err(rustls::Error::InvalidCertificate(
CertificateError::ApplicationVerificationFailure,
));
}
}


Ok(ServerCertVerified::assertion())
}
```

```
}
```

TLS 인증서 요구사항

1. **단일 인증서**: 정확히 하나의 인증서만 제시되어야 함
2. **자체 서명**: 인증서는 자체 서명되어야 함
3. **libp2p 확장**: 공개키 서명을 포함한 유효한 libp2p 확장 필요
4. **상호 인증**: 클라이언트와 서버 모두 인증서 제시 필수

인증 보안 특성

1. 전방향 보안성 (Forward Secrecy)

```
// 각 연결마다 새로운 DH 키 쌍 생성
impl Keypair {
pub(crate) fn new() -> Keypair {
let mut sk_bytes = [0u8; 32];
rand::thread_rng().fill(&mut sk_bytes);
let sk = SecretKey(sk_bytes);
sk_bytes.zeroize(); // 메모리에서 즉시 제거
Self::from(sk)
}
}
```

2. 신원 바인딩 (Identity Binding)

- Identity 키로 DH 공개키에 서명
- 도메인 분리를 통한 서명 재사용 방지
- PeerId와 실제 공개키 간의 암호학적 바인딩

3. 재생 공격 방지

- 각 핸드셰이크마다 고유한 ephemeral 키 사용
- Noise Protocol의 내재적 재생 공격 방지 메커니즘
- 타임스탬프 기반 인증서 유효성 검증 (TLS)

인증 실패 시나리오

1. 서명 검증 실패

```
pub enum Error {
#[error("The signature of the remote identity's public key does not verify")]
BadSignature,
#[error("Authentication failed")]
AuthenticationFailed,
#[error("Remote authenticated with an unexpected public key")]
UnexpectedKey,
}
```

2. PeerId 불일치

```
// 예상된 PeerId와 실제 PeerId가 다른 경우
if remote_peer_id != peer_id {
return Err(rustls::Error::InvalidCertificate(
CertificateError::ApplicationVerificationFailure,
));
}
```

3. 인증서 검증 실패 (TLS)

- 인증서 체인 길이 위반 (1개 초과)
- 자체 서명 검증 실패
- libp2p 확장 누락 또는 무효

성능 고려사항

1. 키 생성 비용

- **Ed25519**: 가장 빠른 키 생성 및 서명/검증
- **RSA**: 상대적으로 느린 연산, 큰 키 크기
- **ECDSA**: 중간 성능, 표준 호환성

2. 핸드셰이크 지연

- Noise XX: 1.5 RTT (3개 메시지)
- TLS 1.3: 1 RTT (최적화된 경우)
- 네트워크 지연이 주요 병목

3. 메모리 사용

```
impl Drop for SecretKey {
fn drop(&mut self) {
self.0.zeroize() // 보안을 위한 메모리 제로화
}
}
```

확장성 및 호환성

1. WebTransport 지원

```
// WebTransport certhashes 확장
pub fn with_webtransport_certhashes(mut self, certhashes: HashSet<Multihash<64>>
self.webtransport_certhashes = Some(certhashes).filter(|h| !h.is_empty());
self
}
```

2. 다중 키 타입 지원

- 런타임 키 타입 감지 및 처리
- 프로토콜 버퍼를 통한 직렬화/역직렬화
- 하위 호환성 유지

3. 프로토콜 네고시에이션

- multistream-select를 통한 보안 프로토콜 선택
- 클라이언트-서버 간 지원 프로토콜 협상
- 폴백 메커니즘 지원

보안 권장사항

1. 키 관리

- Ed25519 키 사용 권장
- 키 재사용 최소화
- 안전한 키 저장 및 로딩

2. 네트워크 보안

- 알려진 피어와의 연결 시 PeerId 사전 검증
- 연결 제한 및 속도 제한 구현
- 악성 피어 차단 메커니즘

3. 구현 보안

- 상수 시간 비교 연산 사용
- 메모리 제로화를 통한 키 누출 방지
- 적절한 에러 처리 및 로깅

결론

Rust libp2p의 피어 인증 시스템은 다음과 같은 특징을 가집니다:

1. **강력한 암호학적 보안**: Noise Protocol과 TLS 1.3 기반

2. **유연한 키 지원**: 다양한 암호화 알고리즘 지원

3. **상호 운용성**: 표준 프로토콜 및 형식 준수

4. **확장성**: WebTransport 등 새로운 전송 프로토콜 지원

5. **성능 최적화**: 효율적인 핸드셰이크 및 메모리 관리

이러한 설계를 통해 libp2p는 분산 네트워크에서 안전하고 효율적인 피어 간 인증을 제공합니다.